

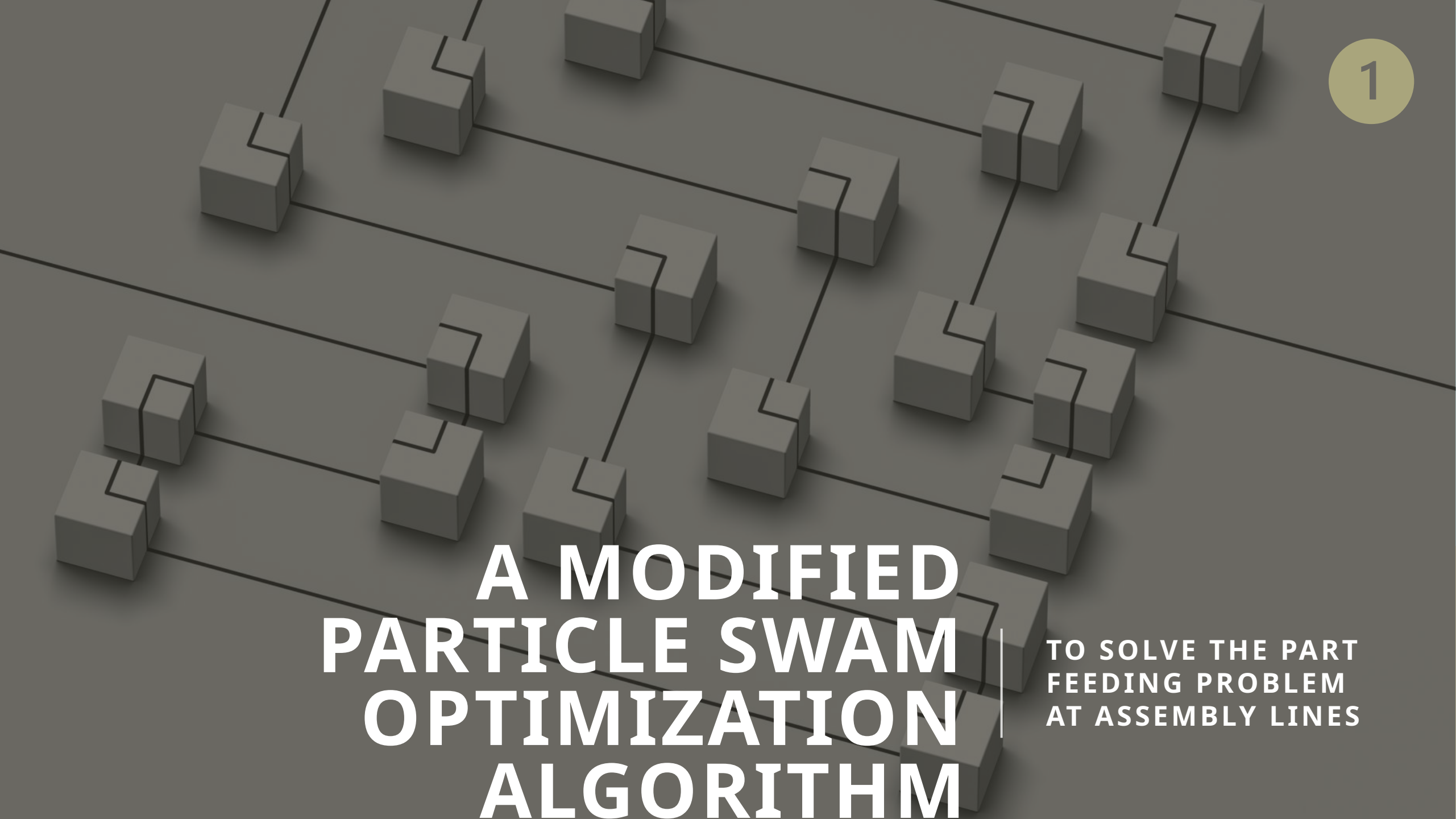
# PROJECT WORK

Engineering optimization

Adhithya Palanasamy
Nuwan Dissanayaka

# CONTENT

1. PART FEEDING PROBLEM
2. ASSEMBLY LINE BALANCING PROBLEM
3. SUPERMARKET LOCATION PROBLEM
4. PRODUCTION PLANNING PROBLEM



1

# A MODIFIED PARTICLE SWAM OPTIMIZATION ALGORITHM

TO SOLVE THE PART FEEDING PROBLEM AT ASSEMBLY LINES

# INTRODUCTION

- Assembly line part feeding problem
- delivery of required part to the assembly workstation in right quantities at the right time.
- Two subproblems
  - Tour scheduling
  - Tow - train loading
- ALPFP has complexity of NP-complete
- we seek heuristic approaches.

# CONSTRAINTS AND OBJECTIVE



There are specific constraints on the tours

number and weight of loaded bins and

replenishment time.

Only full bins are used for delivering parts, and only empty bins are collected

The workstations have limited space, imposing a restriction on the inventory they can hold.



The known part requirements must be fulfilled while respecting the capacities of tours, tow-trains, and workstation storage space..

Primary objective: minimize the number of tours taken.

secondary objective : minimize the sum of part inventories at the workstations.



These objectives are handled using a lexicographic ordering approach, where the primary objective takes precedence over the secondary objective.

# PARTICLE SWARM OPTIMIZATION (PSO)

PSO is Population based stochastic optimisation technique.

Inspired by social behaviour of animals.

It aims to find the optimal solution to a given problem by iteratively updating a population of potential solutions, called particles, based on their own best-known position and the global best position discovered by any particle in the swarm.

The modified PSO (MPSO) introduced here incorporates mutation operators during the position update process

Position :

$$X_i^t = [x_i^t, y_i^t, z_i^t, \cdots]$$

Position update:

$$X_i^{t+1} = X_i^t + V_i^{t+1}$$

Velocity:

$$V_i^{t+1} = wV_i^t + C_1 r_1 (P_i^t - X_i^t) + C_2 r_2 (G^T - X_i^t)$$

**Indirect representation**: represented as permutations of a set of priority rules.

**Two Phase hierarchical construction:**

First Phase – primary objective

Second phase – secondary objective

**Mutation operator:** position of particles (solutions) in the MPSO are updated using mutation operators : swap and insert.

**Fitness evaluation:**

The function defines both defined objective functions into a singe value.

The objective functions are normalized to ensure consistent scaling

# SOLUTION APPROACH (MPSO)

# POSITION AND VELOCITY (REPRESENTATION)

**Position vector** is a sequence of natural number in range [1,M].

Each number corresponds to a priority rule from a set of m rules

These priority rules are used in the solution construction to determine which part number should be assigned to each tow-train on each tour

**Particle velocity:** its influences the particle moment and guide it towards potentially better solution.

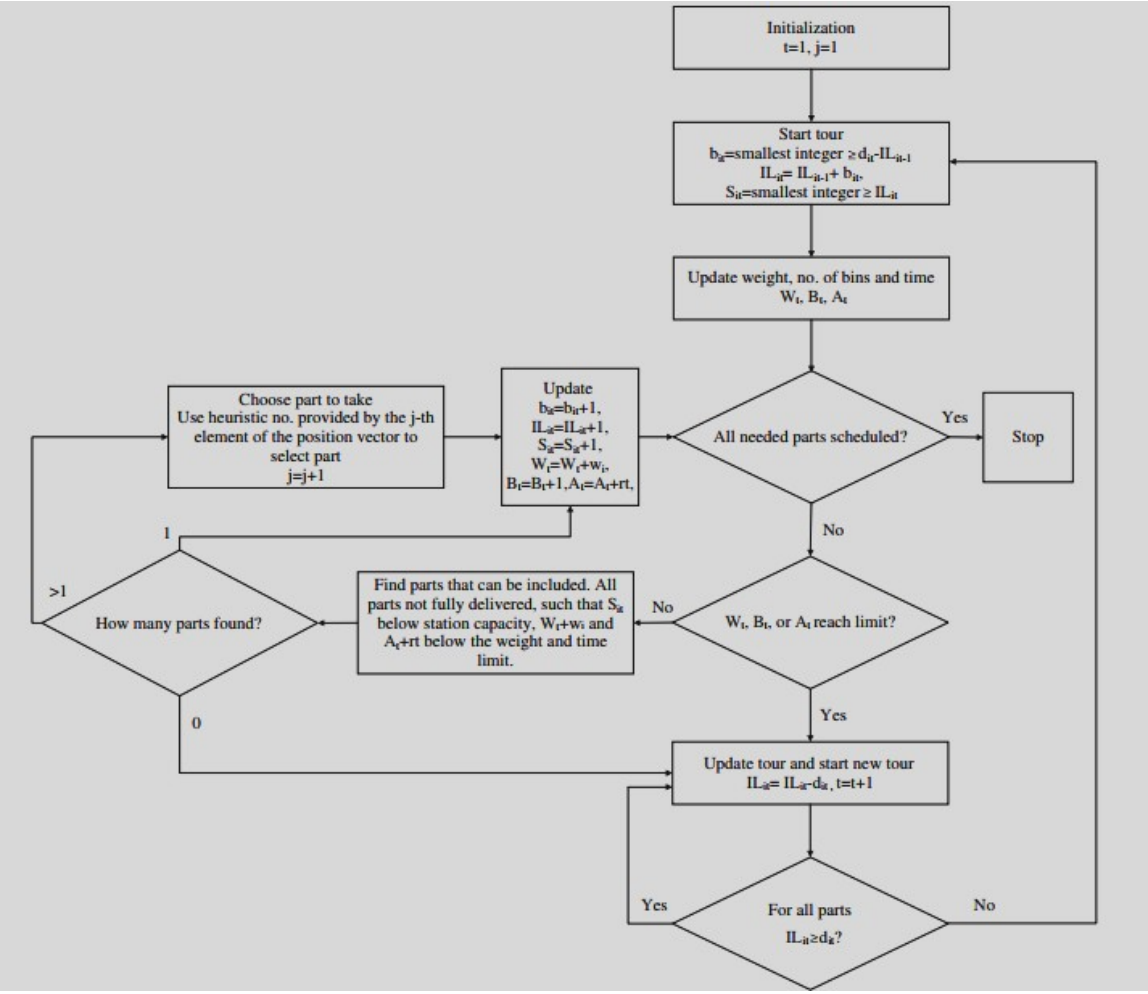
The length of velocity vector is same as the length of particles position vector and has value of real numbers between [-Vmax,Vmax]

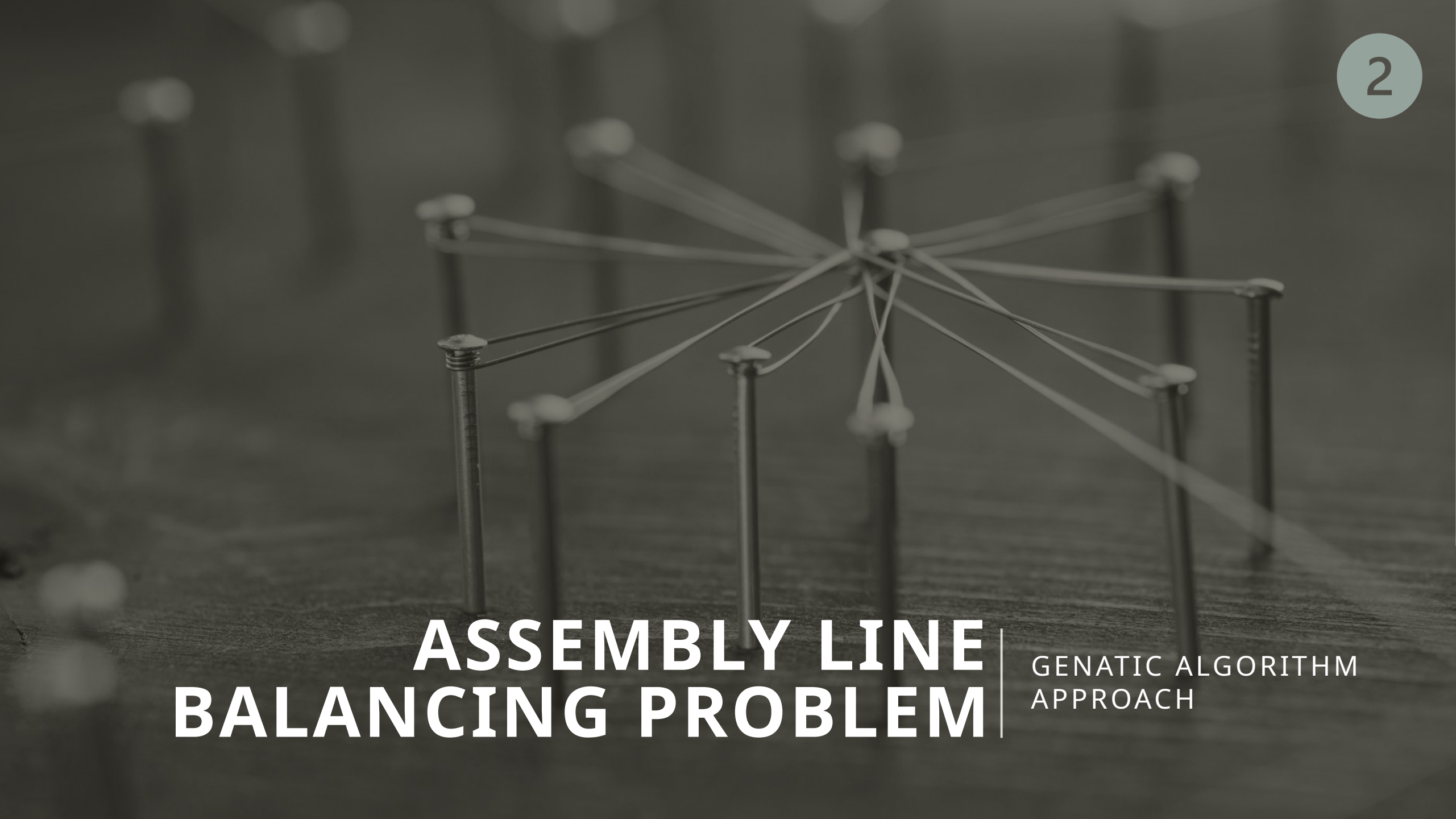
# PARTICLE UPDATING

- Refers to the process of adjusting the position and velocity.
- The particle position is updated through a neighbourhood-based mutation process. This mutation is guided by the particle's velocity, which determines the heuristic to be applied. There are two ways the mutation can occur:
  - by removing a priority rule and replacing it with a new one, or
  - by swapping the current priority rule with another.

$$V_{\mathrm{pk}+1} = W_k \times V_{\mathrm{pk}} + C_1 \times R_1 \times \left(P_{\mathrm{best,p}} - X_{\mathrm{pk}}\right) + C_2 \times R_2 \times \left(G_{\mathrm{best}} - X_{\mathrm{pk}}\right)$$

# PROCEDURE


Initialization
t=1, j=1
Start tour
$b_{it}$=smallest integer ≥ $d_{it}$-$IL_{it-1}$
$IL_{it}$= $IL_{it-1}$+ $b_{it}$.
$S_{it}$=smallest integer ≥ $IL_{it}$
Update weight, no. of bins and time
$W_t$, $B_t$, $A_t$
All needed parts scheduled?
Yes
Stop
No
Update
$b_{ik}$=$b_{ik}$+1,
$IL_{ik}$=$IL_{ik}$+1,
$S_{ik}$=$S_{ik}$+1,
$W_t$=$W_t$+$w_i$,
$B_t$=$B_t$+1, $A_t$=$A_t$+rt,
Choose part to take
Use heuristic no. provided by the j-th element of the position vector to select part
j=j+1
1
>1
How many parts found?
0
Find parts that can be included. All parts not fully delivered, such that $S_{ik}$ below station capacity, $W_t$+$w_i$ and $A_t$+rt below the weight and time limit.
$W_t$, $B_t$, or $A_t$ reach limit?
No
Yes
Update tour and start new tour
$IL_{ik}$= $IL_{ik}$-$d_{ik}$, t=t+1
For all parts
$IL_{it}$≥$d_{it}$?
Yes
No



2

# ASSEMBLY LINE BALANCING PROBLEM

GENATIC ALGORITHM APPROACH

# GENATIC ALGORITHM APPROCH-OPTIMIZATION

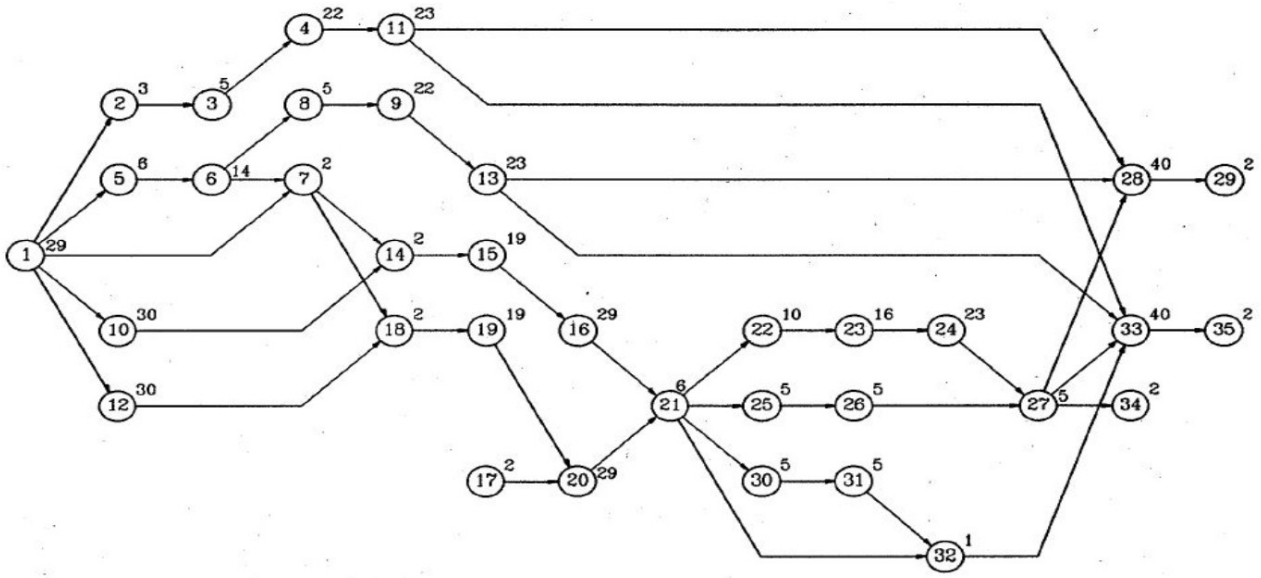
Inspired by biological evolution and natural selection.
Used in optimization problems to find the best solution among a large set of possibilities

- Initialization
- Fitness Evaluation
- Selection
- Crossover
- Mutation
- Replacement
- Termination.

By repeatedly applying these steps, genetic algorithms explore the search space, gradually improving the quality of solutions over generations. The algorithm's ability to maintain a diverse population and efficiently explore the solution space makes it effective in finding near-optimal or optimal solutions to complex optimization problems.

# PROBLEM INTRODUCTION

- Assemble a product consists of 35 parts
- Assembly operation follow sequential order due to technical constraints
- Maximum acceptable cycle time is 61 seconds



# PARAMETERS

- Crossover rate
- Mutation rate
- Crossover type
- Population Size
- Generations

# REVERSE ORDER CROSSOVER (ROX)

N = 100

Crossover probability= 80%

Swap mutation

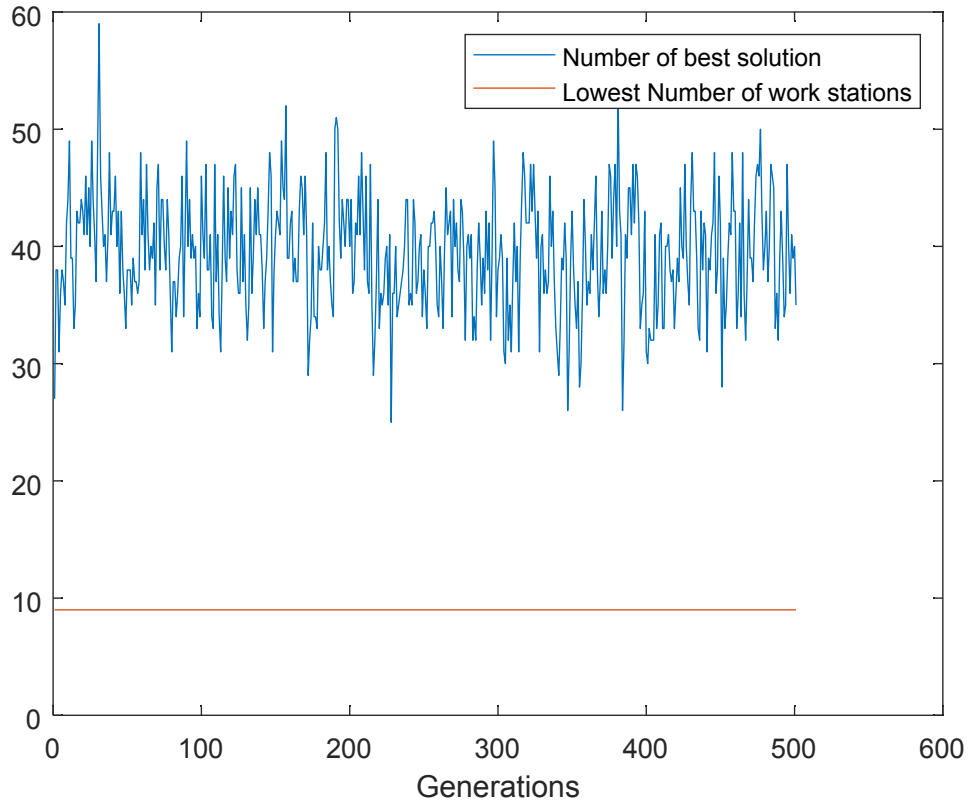
Best result = 9 work station

work load deviation = 7,111

Population with 9 workstations = 39

Population with 10 work station = 60

Population with 11 work station = 1



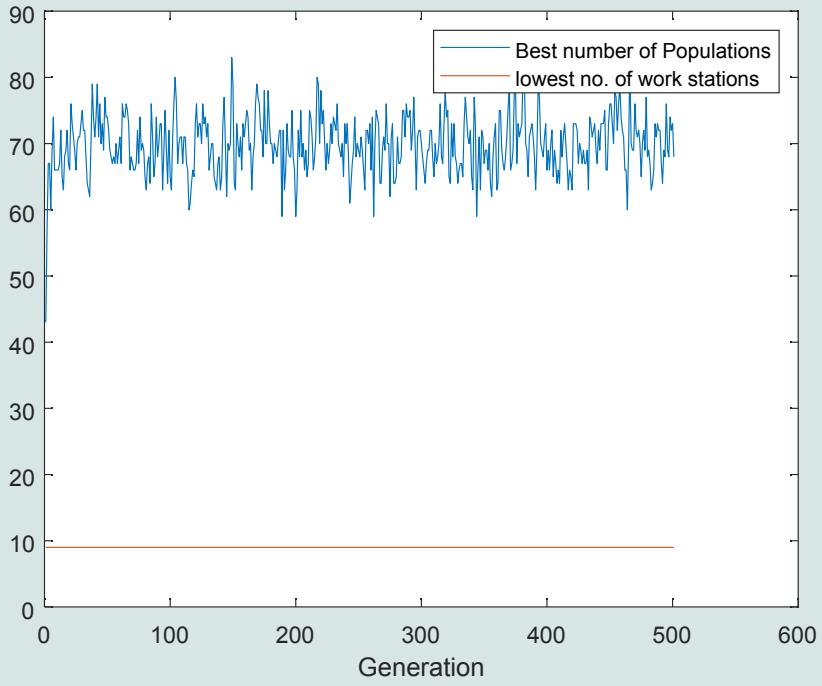
# CROSSOVER PROBABILITY

Crossover probability= 50%

Population with 9 workstations = 68

Population with 10 work station = 31

Population with 11 work station = 1

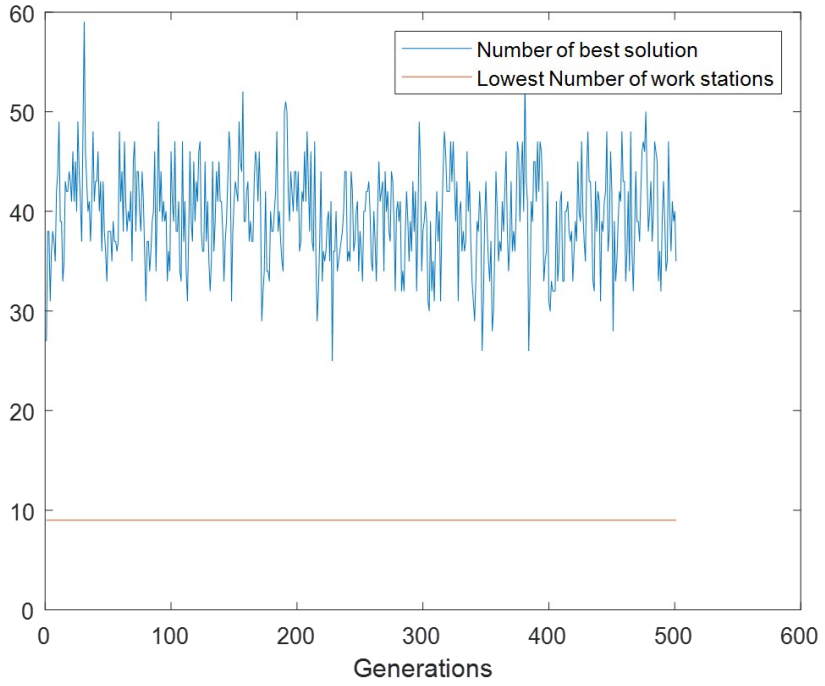



Crossover probability= 80%

Population with 9 workstations = 39

Population with 10 work station = 60

Population with 11 work station = 1

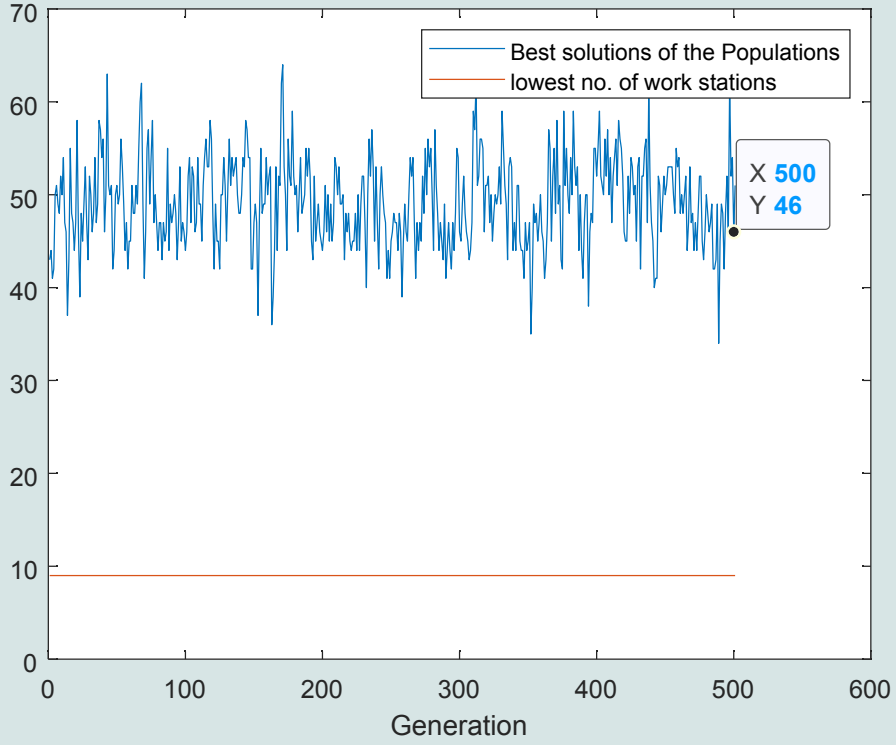


# MUTATION PROBABILITY

Mutation probability = 1%

Swap mutation

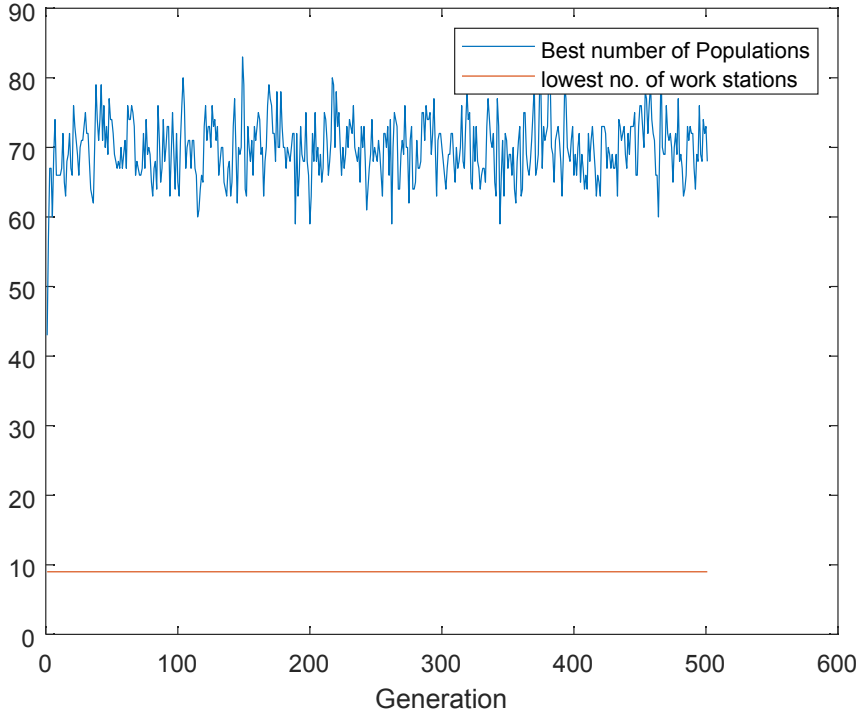
Population with 9 workstations =46




Mutation probability = 10%

Swap mutation

Population with 9 workstations = 68



# UNIFORM CROSSOVER

N = 100

Crossover probability= 50%

Swap mutation

Best result = 9 work station

work load deviation = 7,111

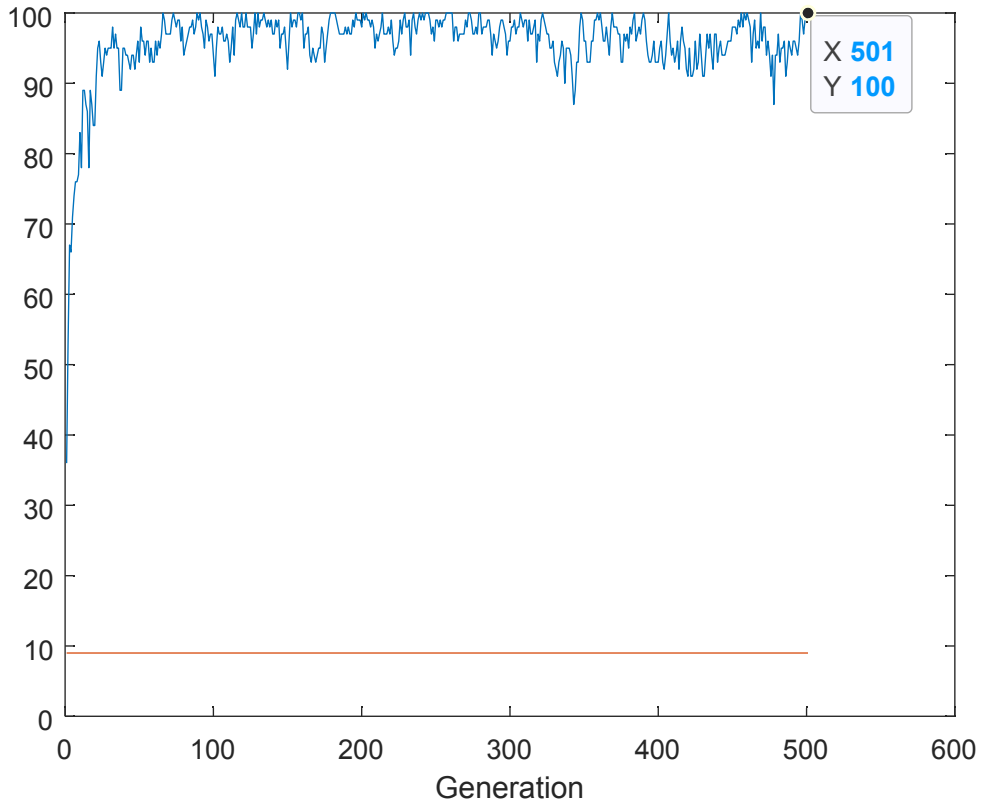
Population with 9 workstations = 100

population with 10 work station = 0

Population with 11 work station = 0

Converges fast

Better diversity for this problems



# TWO POINT CROSSOVER

N = 100

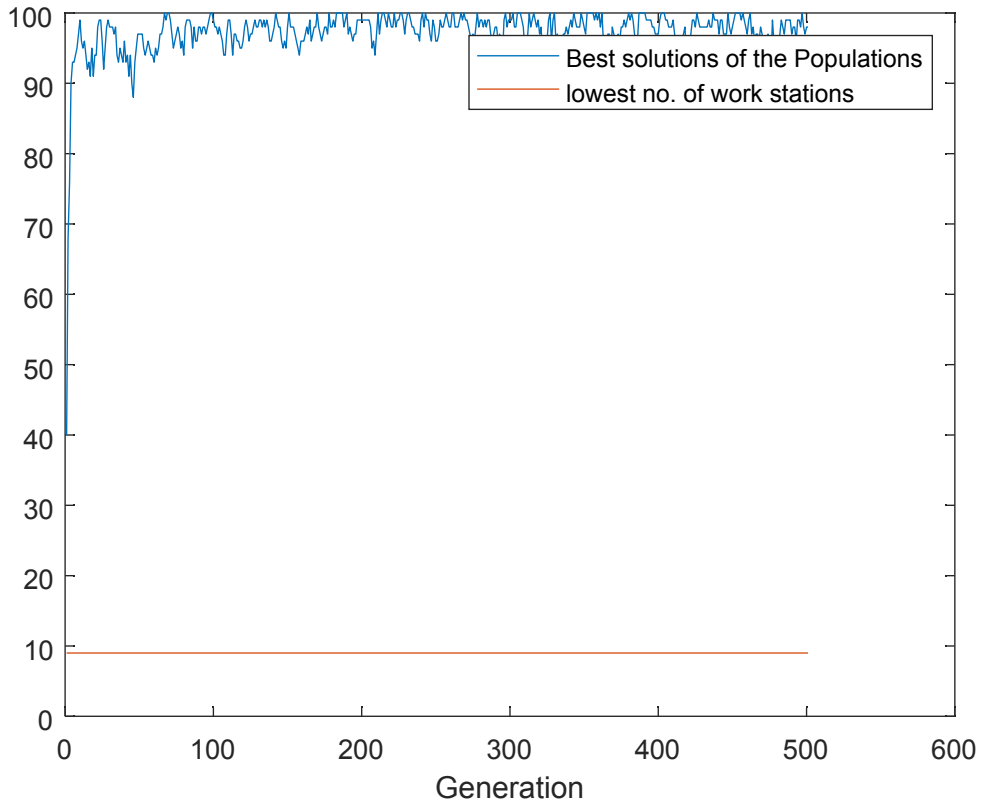
Crossover probability= 50% Swap mutation

Best result = 9 work station

Population with 9 workstations = 100

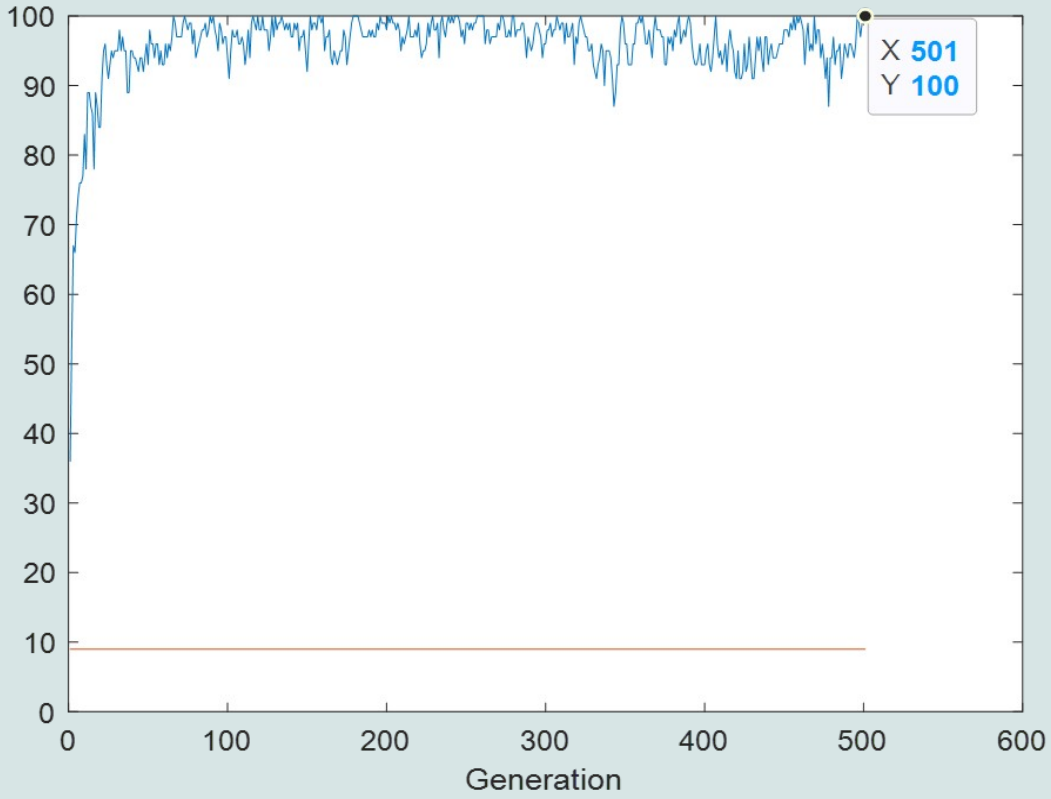
population with 10 work station = 2

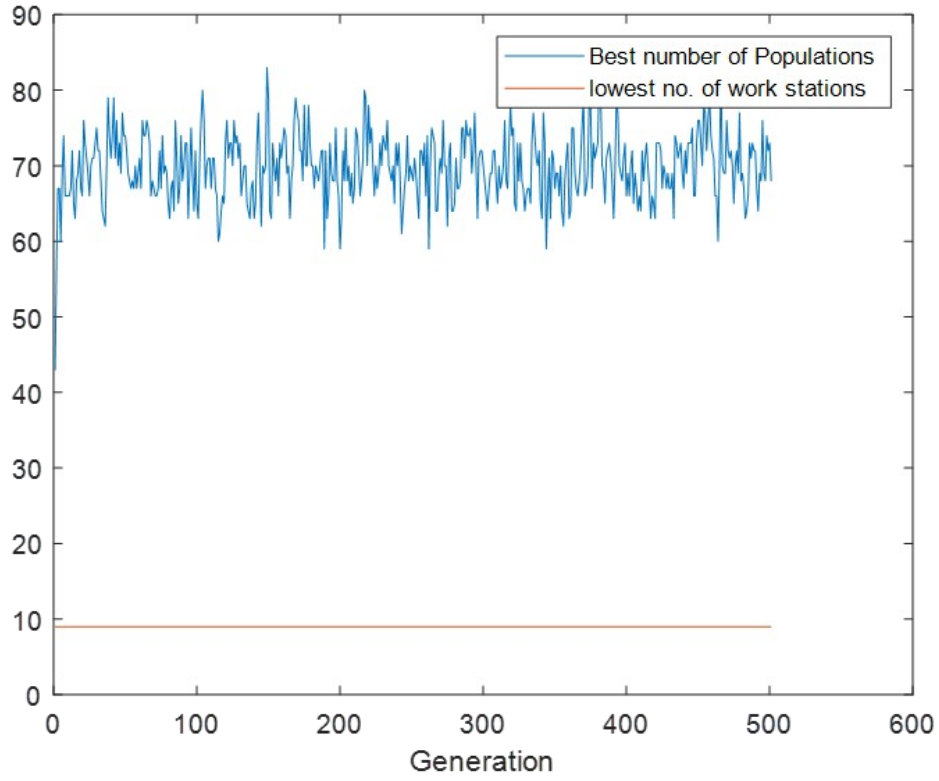
Population with 11 work station = 0



# UNIFORM CROSSOVER VS REVERSE ORDER CROSSOVER

MUTATION PROBABILITY = 10%  POPULATION SIZE = 100

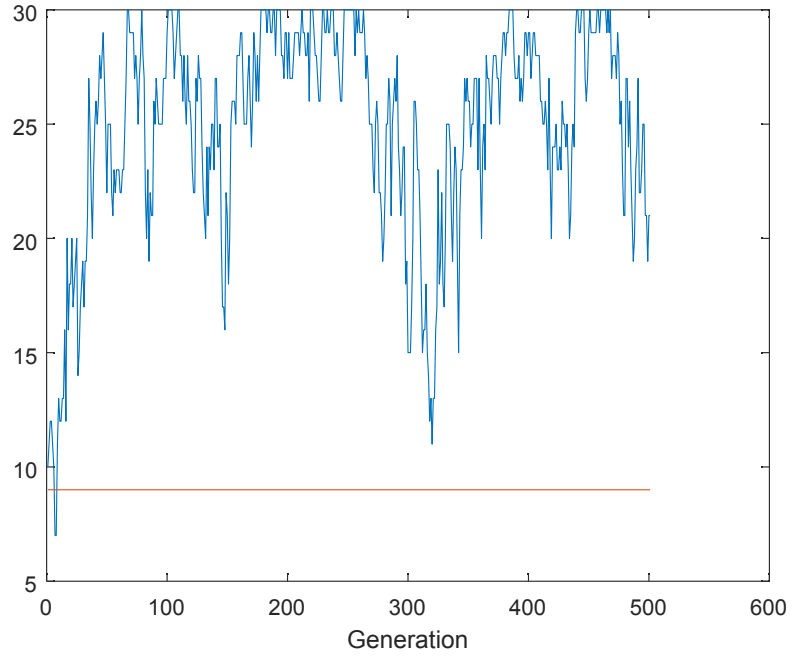


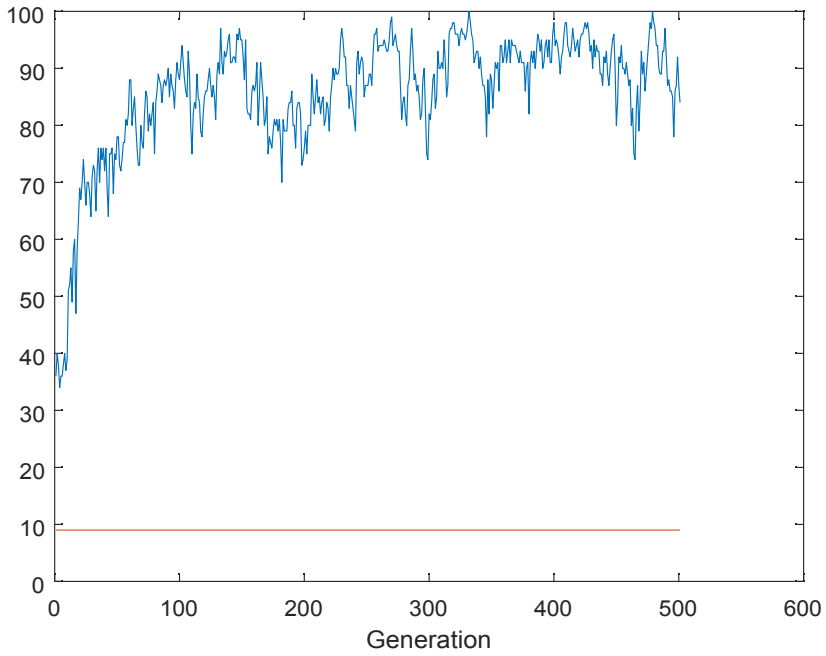
# POPULATION SIZE

- A larger population size typically means that more candidate solutions will be evaluated in each generation, which can increase the diversity of the population and the likelihood of finding better solutions.
- Also larger N lead to high complexity

N= 30




N = 100



# CONCLUSION

higher crossover probability leads to a higher likelihood of producing offspring that are similar to their parents. On the other hand, a lower crossover probability leads to a higher likelihood of producing offspring that are more diverse from their parents, usually between 0,5 – 0,8

Mutation rate determines evolution along the generation, it allows to escape local optimum. Usually 0,05 – 0,1

The effectiveness of a crossover operator can depend on the specific problem being solved and the characteristics of the problem instance and the genetic algorithm being used.

ROX and partially mapped crossover would be not a good choice for this problem but uniform, two-point, single point crossovers are better options

Additionally type of Selection would also help to obtain better results.



3

# SUPERMARKET LOCATION PROBLEM

SIMULATED ANNEALING APPROACH

# INTRODUCTION TO THE PROBLEM

01

The assembly line consists of 30 workstations and demand of each assembly station is known

02

Assembly stations are fed with Line-integrated supermarkets and the capacity of each is known

03

Coordinates of workstations and line-integrated supermarkets are given

- The supermarket installation cost - £1000
- Shipment unit cost - £1

| WS | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Dem | 7 | 7 | 10 | 1 | 7 | 9 | 4 | 5 | 4 | 6 | 6 | 3 | 9 | 1 | 5 | 3 | 5 | 9 | 6 | 3 | 5 | 9 | 9 | 9 | 9 | 3 | 9 | 5 | 8 | 10 |
| Xw | 0 | 2 | 4 | 6 | 8 | 10 | 12 | 14 | 16 | 18 | 20 | 22 | 24 | 26 | 28 | 30 | 32 | 34 | 36 | 38 | 40 | 42 | 44 | 46 | 48 | 50 | 52 | 54 | 56 | 58 |
| Yw | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

| SUP | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| Capacity | 60 | 60 | 60 | 60 | 60 |
| Xs | 5.8 | 17.4 | 29 | 40.6 | 52.2 |
| Ys | 5 | 5 | 5 | 5 | 5 |

# SIMULATED ANNEALING - OPTIMIZATION

Inspired by the physical process of annealing in metallurgy.
Used to solve optimization problems where the search space is large and complex.

- Initialization:
- Define a temperature schedule
- Perturbation.
- Evaluation.
- Acceptance criterion:
- Update.
- Cooling.
- Termination.

$$P = \exp(-\Delta Z / T)$$

Simulated annealing's ability to explore the search space by accepting worse solutions at the beginning and transition to exploitation as the temperature decreases allows it to escape local optima and find near-optimal or optimal solutions. It is particularly effective in problems where the search space is rugged or has many local optima.

# PARAMETERS CONSIDERED

- Cooling rate
- Cooling function
- Initial temperature (T0)
- Inner loop

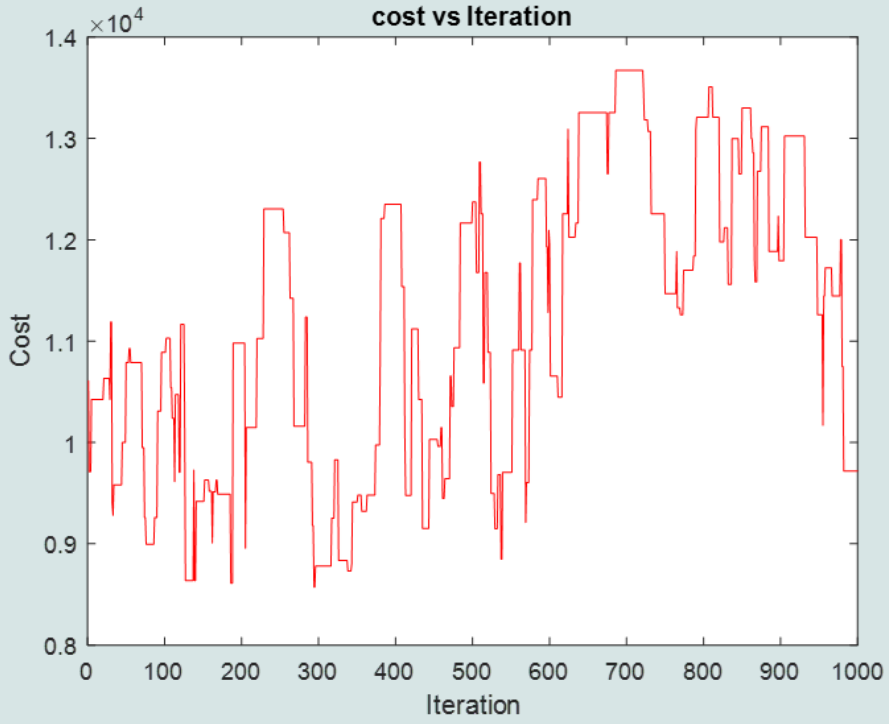
# LINEAR COOLING

T(k+1) = T0 – alpha*K

Alpha = 1

Cost = 9772

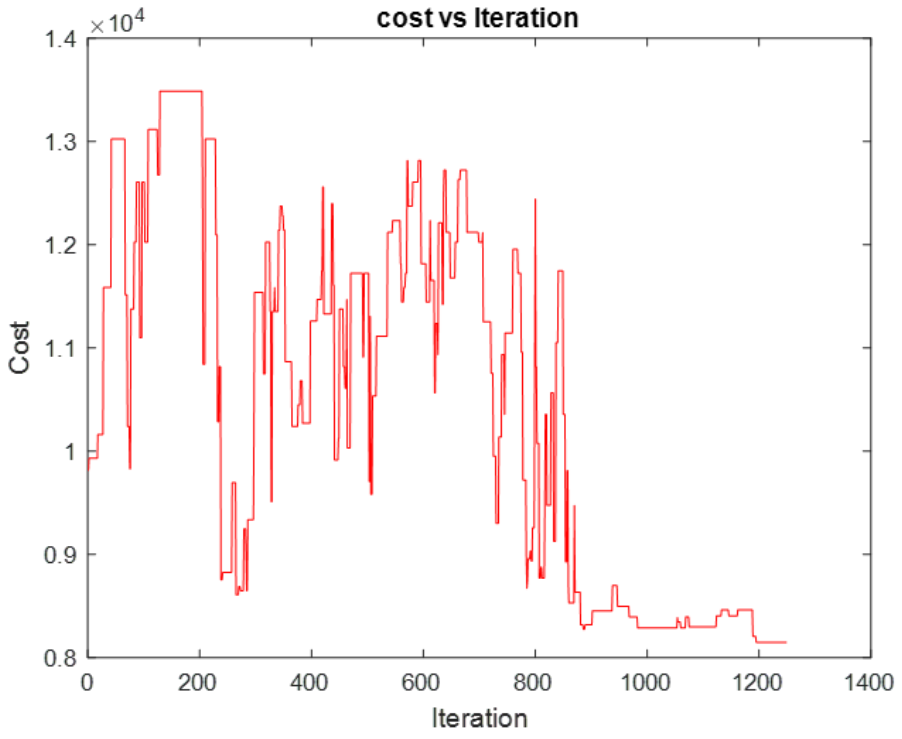
Iteration = 1000




Alpha = 0,8

Cost = 8140

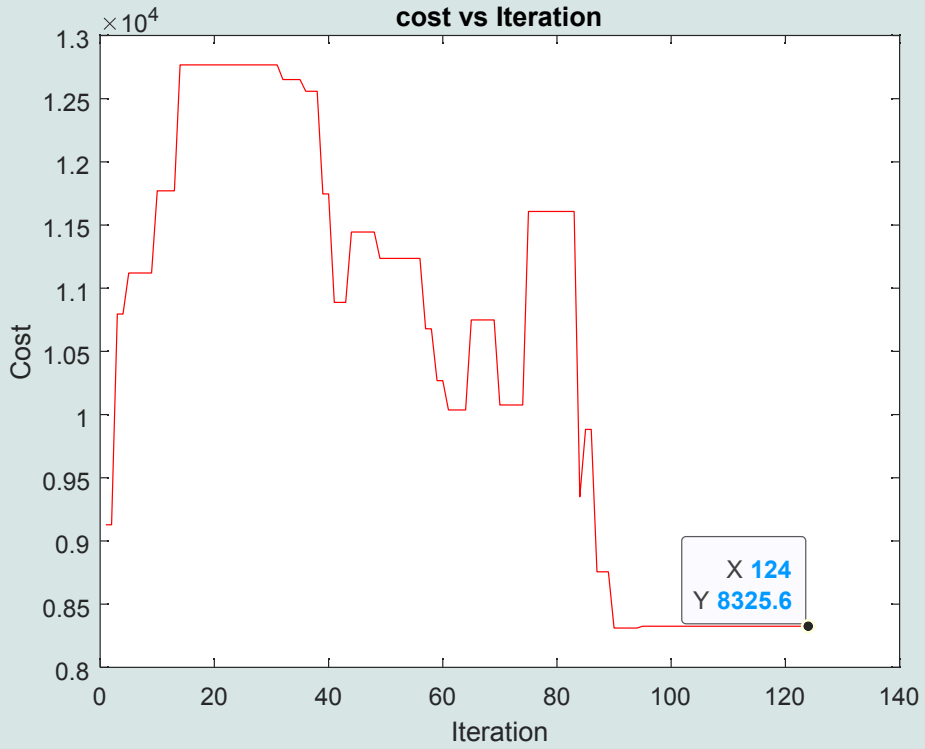
Iteration 1249



# INITIAL TEMPERATURE

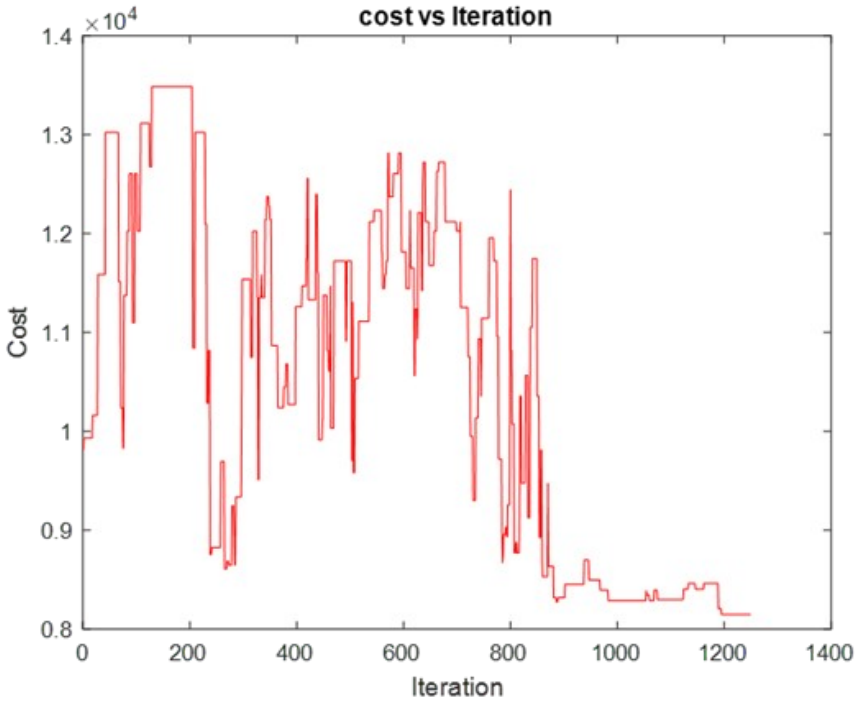
Cost = 8325

T0 = 100




cost = 8140

T0 = 1000 & alpha = 0,8



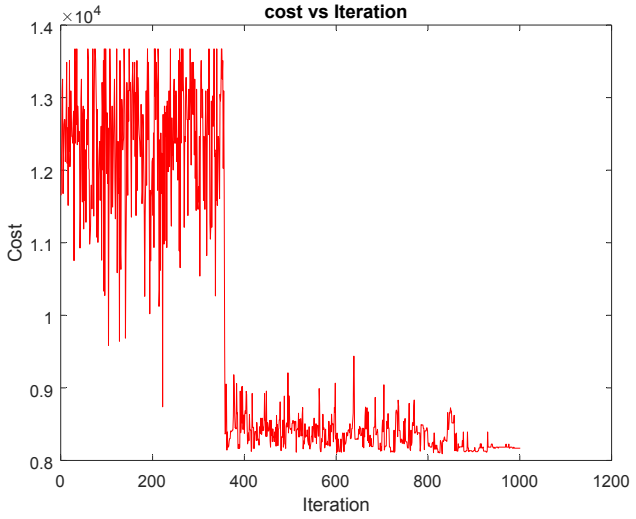
# LINEAR COOLING WITH INNER LOOP

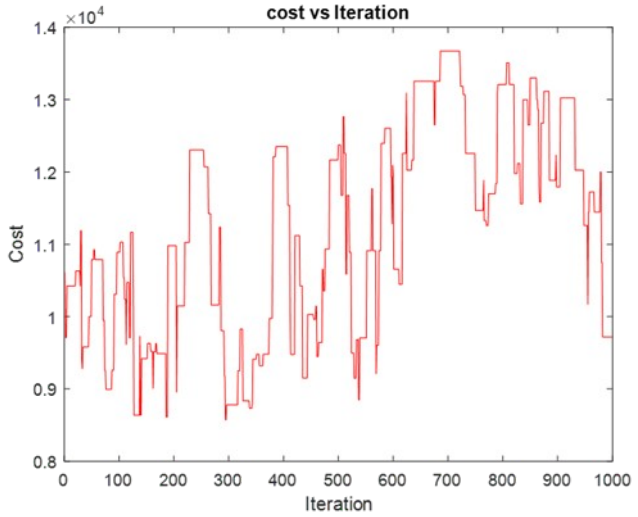
**Inner loop** = 50

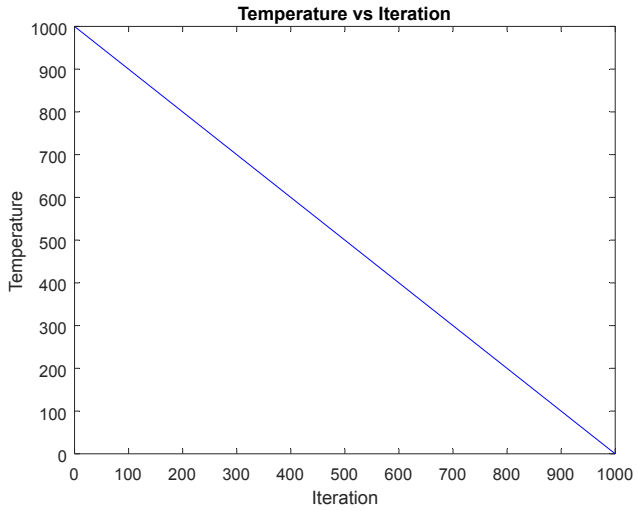
**alpha** = 1

**Cooling function =** T0 – alpha*K

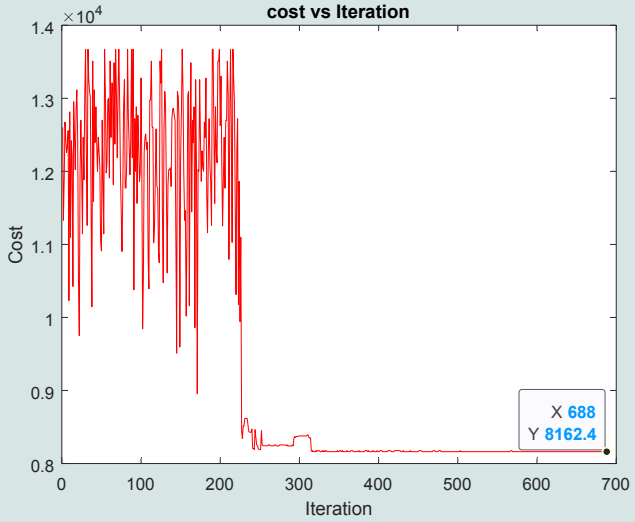
**cost = 8168**



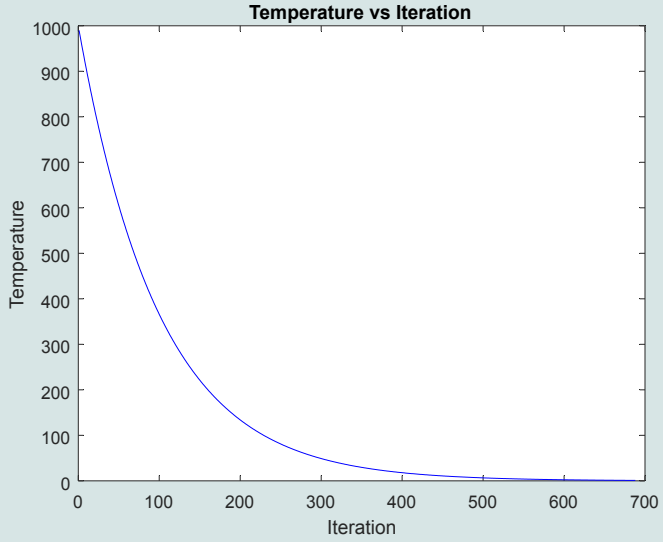
# EXPONENTIAL COOLING

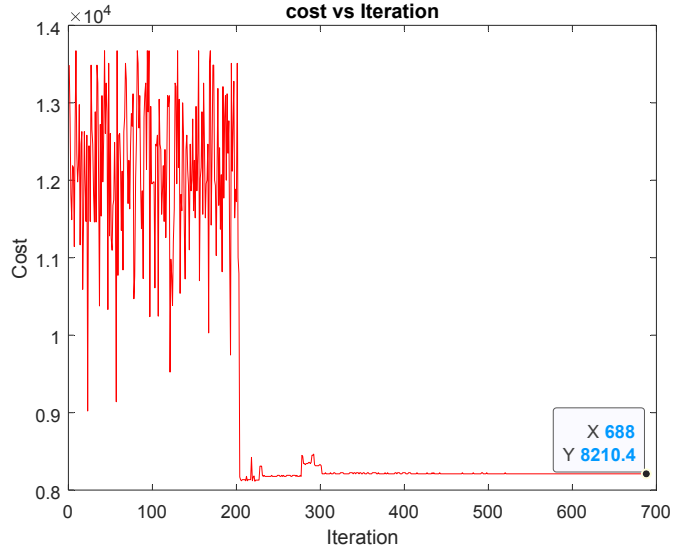



T(k+1) = T * alpha

alpha = 0,99

Cost = 8162

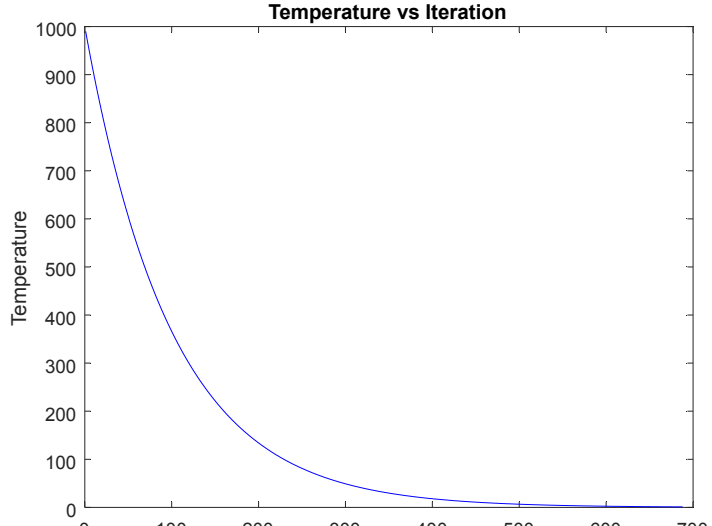



T(k+1) = T0 * alpha^k

Alpha = 0,99

Cost = 8210

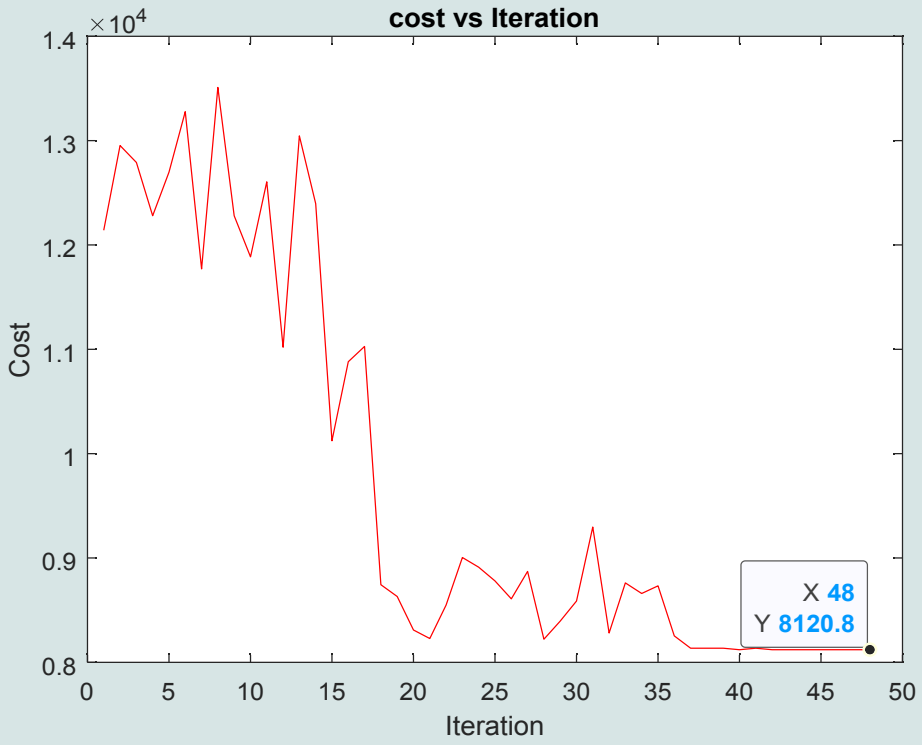


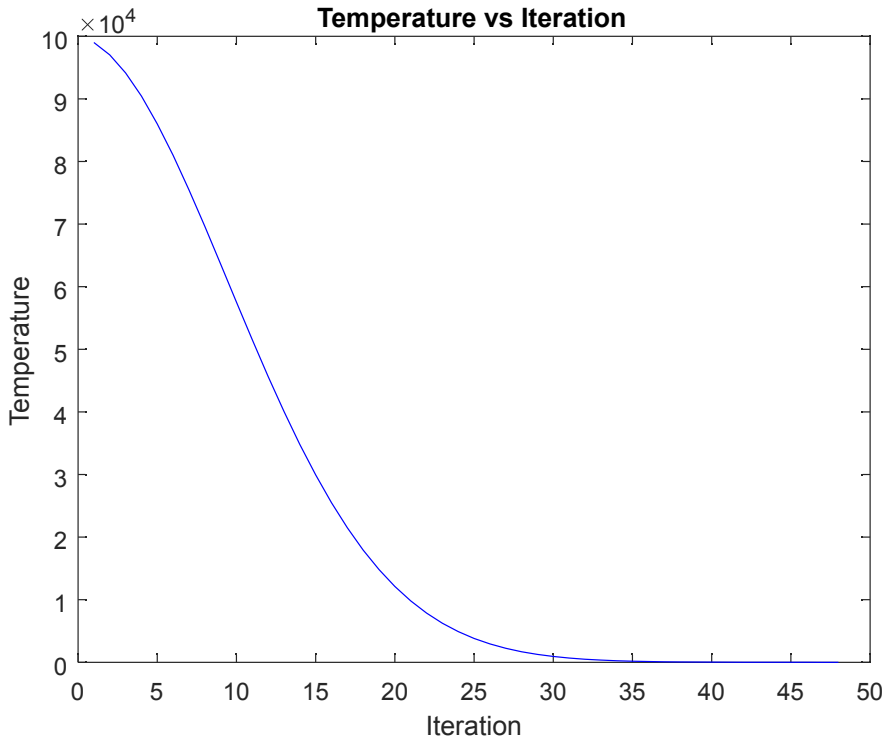
# HYPERBOLIC COOLING

Cost = 8120

Higher starting temperature

Higher inner loops



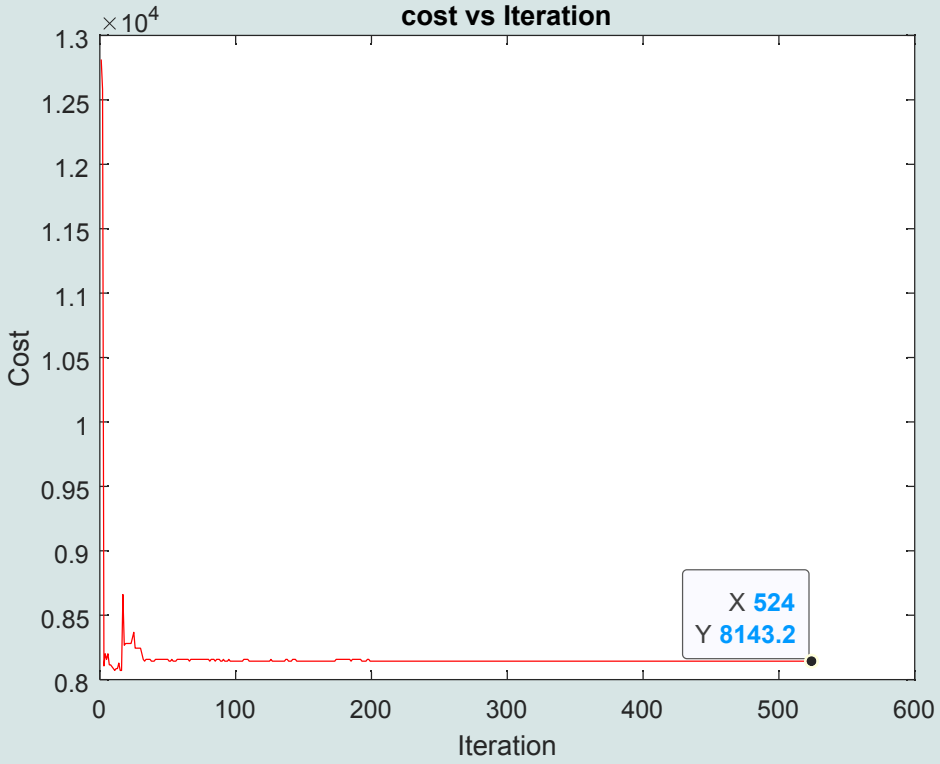
# FAST COOLING

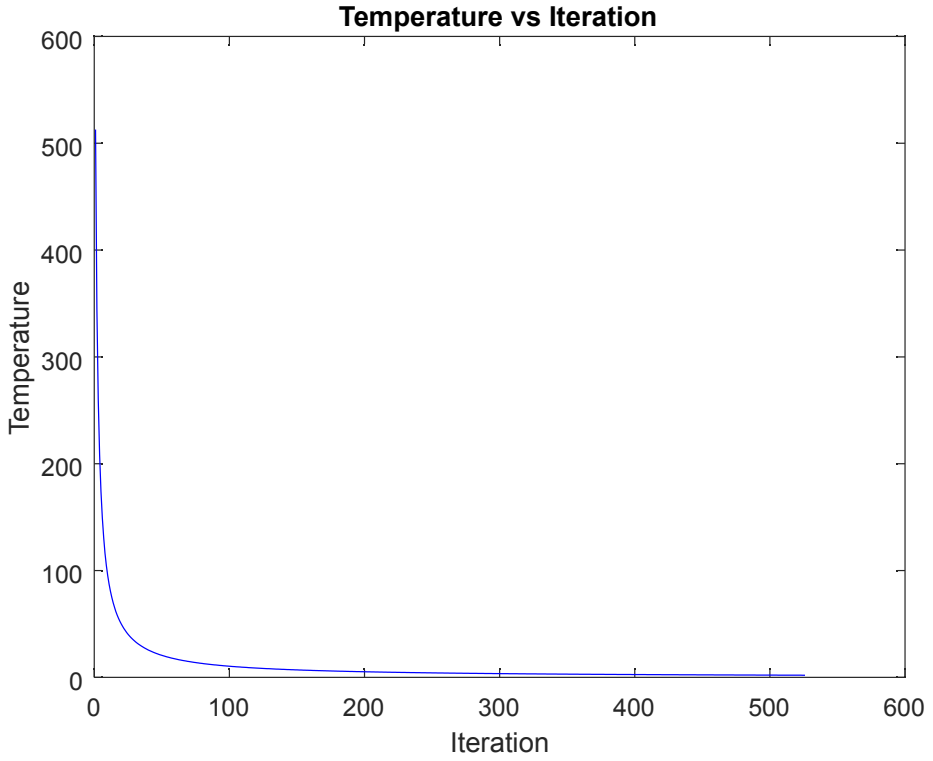
T0 = 1000

T(k+1) = T0/ (1 + alpha *k)

K – iteration

Alpha = 0,95



# MUTATION FOR EXPLOITATION

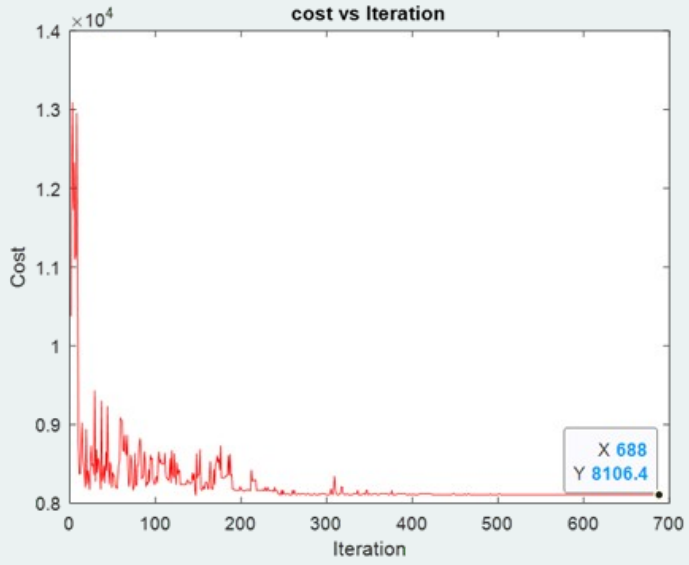
Exponential cooling

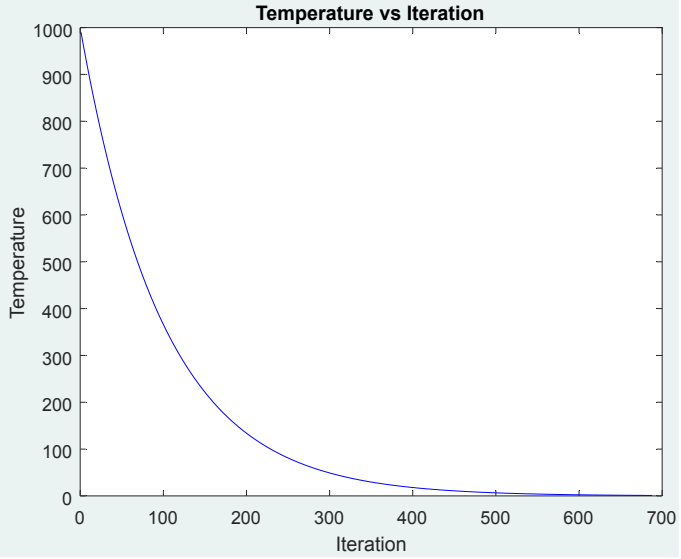
Reverse mutation

Alpha = 0,99

Converges fast and reaches global optimum quicker

Cost = 8106 ; inner loop = 50

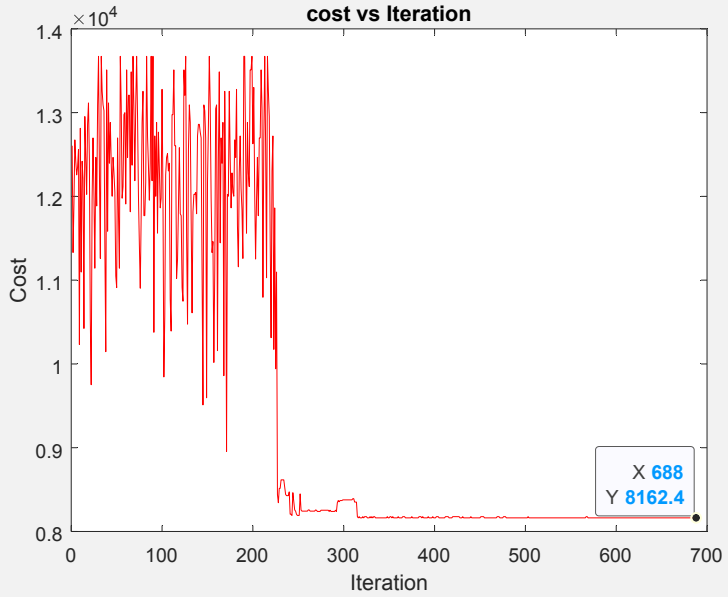



Exponential cooling

Alpha = 0,99

Cost = 8162 ; inner loop = 50



# SWARM ALGORITHM APPROACH

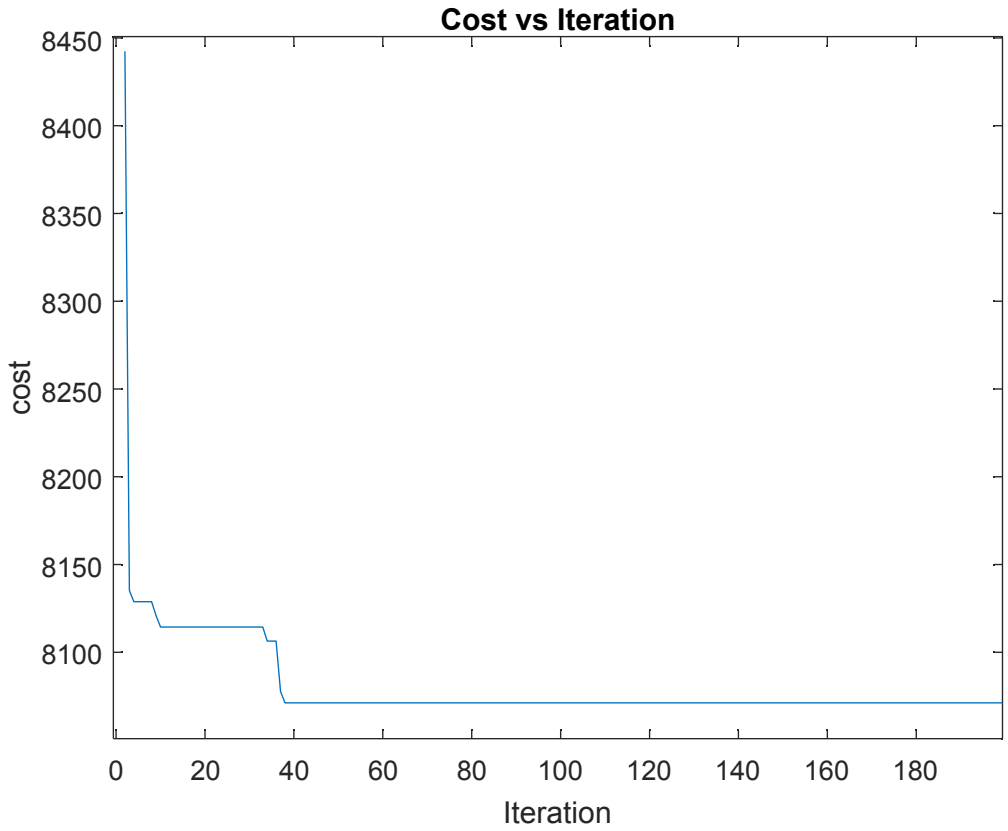
Cost = 8071.2

Supermarket shipment = [41 28 32 41 44]

C1, C2 = 1

W = 0.9 – 0.4

Max iteration = 200



# CONCLUSION

Cooling rate: 0,8< alpha < 1, determines the pace of the cooling cycle.

Cooling function- mathematical formula that determines how the temperature decreases over time during the annealing process. Slower cooling more time to explore but also time consuming , while faster cooling is the vise versa.

Inner loops provides more time to explore and takes the best value of it to the next temperature.

Initial temperature and final temperature depends on the cooling function. Best parameters can be obtained by trail.

Mutation provides additional support to acceptance probability by escape local optimum by shifting the search space to avoid premature convergence.



4

# PRODUCTION PLANNING PROBLEM CASE OF STEELMAKER

Linear programming

# LINEAR PROGRAMMING

Exact solution Methods

A steel maker produces girders in four sizes: small, medium, large and extra-large

three types of machines A, B, and C,

The company receives a weekly order to produce girders of length 12,000, 8000, 6000, and 6000 metres for small, medium, large, and extra-large size girders, respectively.

Objective is to minimize the operational cost, given the available time and cost to operate the machines.

| Girder size | Machine | | |
|---|---|---|---|
| | A | B | C |
| Small | 300 | 600 | 800 |
| Medium | 250 | 400 | 800 |
| Large | 200 | 350 | 600 |
| Extra-large | 100 | 200 | 300 |

# INPUTS



The lengths (in meters) of the girder of each size that can be produced per hour by each machine are given in the table below.



The operational costs of machines A, B, and C amount to $30, $50, and $80 per hour respectively.



The weekly contract to produce girders of size small, medium, large, and extra-large size girders is 12,000, 8000, 6000, and 6000 meters respectively.



The available time in hours for machines to operate in a week is 50,30 and 20 for machines A, B and C respectively.



We assume that each machine can only produce one size of girders at a time

# DECISION VARIABLES

Decision variables : quantities that can be adjusted to achieve the optimal value of the objective function subject to the constraints.

In this scenario the decision variables that determine the optimality of our objective is the number of hours each machine operates to produce each size of girders which is denoted as $x_{ij}$.

$i \in \{1, 2, 3, 4\}$ denotes girder of four distinct sizes, namely small, medium, large, and extra-large.

$j \in \{1, 2, 3\}$ denotes machines, A, B, and C respectively.

| | A | B | C |
|---|---|---|---|
| Small | $x_{11}$ | $x_{12}$ | $x_{13}$ |
| Medium | $x_{21}$ | $x_{22}$ | $x_{23}$ |
| Large | $x_{31}$ | $x_{32}$ | $x_{33}$ |
| Extra-Large | $x_{41}$ | $x_{42}$ | $x_{43}$ |

# OBJECTIVE FUNCTION

$$\boldsymbol{Min\ Z} = \sum_{i=1}^{4}(30\ x_{i1}) + \sum_{i=1}^{4}(50\ x_{i2}) + \sum_{i=1}^{4}(80\ x_{i3})$$

# CONSTRAINTS

$x_{ij} \geq 0, \ \forall \ \mathrm{i} \in \{1,2,3,4\}, \ \ \mathrm{j} \in \{1,2,3\}$, Time can not be negative.

Total hours machine A,B and C operate should not exceed 50, 30 and 20

$$\sum_{1=1}^{4} x_{i1} \leq 50$$

$$\sum_{1=1}^{4} x_{i2} \leq 30$$

$$\sum_{1=1}^{4} x_{i3} \leq 20$$

# CONSTRAINTS

The number of girders to be produced in metres of size small, medium, large and extra-large should not be less than 1200, 8000, 6000 and 6000.

$$
\begin{aligned}
300x_{11} + 600x_{12} + 800x_{13} &\geq 12000 \\
250x_{21} + 400x_{22} + 800x_{23} &\geq 8000 \\
200x_{31} + 350x_{32} + 600x_{33} &\geq 6000 \\
100x_{41} + 200x_{42} + 300x_{43} &\geq 6000
\end{aligned}
$$

# SOLUTION OBTAINED THROUGH EXCEL SOLVER

Total operational cost :

(Z) = 30 (10 +30 ) + 50 ( 15 + 15) + 80 (10 + 10) = $4300

| | A | B | C |
|---|---|---|---|
| Small | 10 | 15 | 0 |
| Medium | 0 | 0 | 10 |
| Large | 30 | 0 | 0 |
| Extra-Large | 0 | 15 | 10 |
| **Total Hours** | 40 | 30 | 20 |

# SOLUTION OBTAINED THROUGH MATLAB

The data fed is in form of matrix

where the column concerns the variables, and its constraints coefficient. Columns representing the decision variables was considered differently in both type, which is as below.

Type 1 : [ $x_{11}$ $x_{21}$ $x_{31}$ $x_{41}$ $x_{12}$ $x_{22}$ $x_{32}$ $x_{42}$ $x_{13}$ $x_{23}$ $x_{33}$ $x_{43}$ ]

Type 2 : [ $x_{11}$ $x_{12}$ $x_{13}$ $x_{21}$ $x_{22}$ $x_{23}$ $x_{31}$ $x_{32}$ $x_{33}$ $x_{41}$ $x_{42}$ $x_{43}$ ]

The matrix constructed for constraints and objective function is different in both types.

# RESULT FORM MATLAB

Type 1

| | A | B | C |
|---|---|---|---|
| Small | 10 | 15 | 0 |
| Medium | 0 | 0 | 10 |
| Large | 30 | 0 | 0 |
| Extra-Large | 0 | 15 | 10 |
| **Total Hours** | 40 | 30 | 20 |

(Z) = 30 (10 +30 ) + 50 ( 15 + 15) + 80 (10 + 10)

= $4300

Type 2

| | A | B | C |
|---|---|---|---|
| Small | 0 | 20 | 0 |
| Medium | 0 | 0 | 10 |
| Large | 0 | 0 | 10 |
| Extra-Large | 40 | 10 | 0 |
| **Total Hours** | 40 | 30 | 20 |

(Z) = 30 (40 ) + 50 ( 20 + 10) + 80 (10 + 10)

= $4300

# DISCUSSION

Though the decision values are different to satisfy the constraints, the actual variable that determine the objective function is the total hours every machine has operated, and it is the same for every method which is 40, 30 and 20 hours for machines A, B and C respectively.

Different formulations of the same linear programming problem may have different constraints or different ways of expressing the objective function, even though they represent the same problem. As a result, different formulations may lead to different values of the decision variables that satisfy the constraints and achieve the optimal value of the objective function.

The optimal value of the objective function should be the same for all formulations of the same linear programming problem because the objective function is a linear combination of the decision variables and their coefficients, and the coefficients do not change between different formulations of the same problem.

# CONCLUSION



Obtain the exact optimal solution for a problem



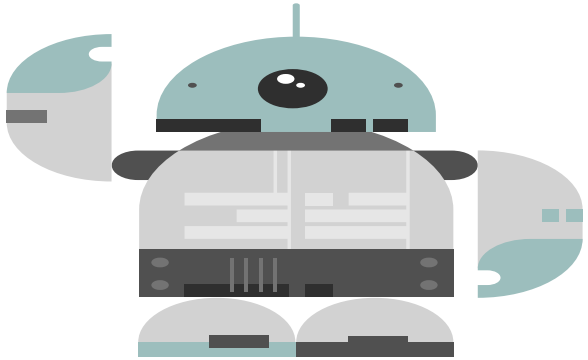
Suitable only if there is  linearity in the problem. Many real word, problems involve non-linear relationships or objectives, which cannot be accurately modeled using linear programs



Difficult to construct for problems with higher dimension and constraints, and also not feasible as it is time consuming, becomes computationally expensive.



Requires deep knowledge on the problem to formulate



# THANK YOU